Diagnosing the Remaining Night False Alarms — Sea Surface Artifacts

Introduction

The simple physical check eliminated 54,542 false alarm pixels (a 63.3% reduction) in one shot. That confirms the initial diagnosis: the majority of the spike was driven by cold cloud-edge refraction.

Now let's examine why 31,564 false alarms still slipped through.

1. The Physics of the Remaining 31,564 Pixels

Recall the coordinates from your benchmark incident report:

- Front #1: Lat 37.5000°N, Lon 3.4180°E
- Front #2: Lat 37.5000°N, Lon 5.7700°E
- Front #3: Lat 37.4781°N, Lon 6.1060°E

In Algeria, any latitude north of 36.9°N is open water—the Mediterranean Sea.

```
37.5°N ~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~  Still passing the check!
          Mediterranean Sea (Summer SST: 293 K - 298 K)
36.9°N  Algerian Coastline
          Tell Atlas Mountain Forests (True Land Fires)
34.0°N
```

During June, July, and August, the Mediterranean Sea Surface Temperature (SST) sits between 20°C and 25°C (293 K to 298 K).

Because 295 K > 285 K, our basic temperature gate ($T_{3.9} \geq 285$ K) allowed all sea surface pixels to pass. The model is flagging thermal gradients on the sea surface where wildfires cannot physically exist.

2. Inspecting the Remaining Errors (`inspect_night_false_alarms`

Before modifying any model files, let's measure two specific factors on the 100 night test scenes:

1. **The Land Boundary:** How many of the remaining 31,564 false positives sit in open water (Latitude > 36.9°N)?

2. **Threshold Sensitivity:** What happens when we test $\tau = 0.90$ and $\tau = 0.95$, just as we did for the daytime model?

Create `inspect_night_false_alarms.py` in `D:\ramy uni\AI ENGENEERING\FINDFIRE\findfire\`:

```
import os
import numpy as np
import pandas as pd
import torch
import torch.nn.functional as F

from unet_model import UNet

INDEX_PATH = r"D:\FINDFIRE_DATA\processed_800\dataset_index_800.
    csv"
NIGHT_CKPT = os.path.join("checkpoints", "best_unet_night_800.pth
    ")
device = torch.device("cuda" if torch.cuda.is_available() else "
    cpu")

# Load trained night model
checkpoint = torch.load(NIGHT_CKPT, map_location=device,
    weights_only=False)
model = UNet(in_channels=3, out_channels=1, base_features=64).to(
    device)
model.load_state_dict(checkpoint["model_state_dict"])
model.eval()

# Filter for the 100 night test scenes
df = pd.read_csv(INDEX_PATH)
df["hour"] = pd.to_datetime(df["timestamp"]).dt.hour
night_test = df[
    (df["split"] == "test") & ((df["hour"] < 6) | (df["hour"] >
        18))
].reset_index(drop=True)

print(f"Analyzing {len(night_test)} Night Test Scenes...")

means = torch.tensor([295.0, 285.0, 10.0]).view(3, 1, 1)
stds = torch.tensor([15.0, 12.0, 8.0]).view(3, 1, 1)

# Grid parameters
LAT_MIN, LAT_MAX = 34.0, 37.5
GRID_H = 120

# Latitude vector for the 120 rows
lat_per_row = np.linspace(LAT_MAX, LAT_MIN, GRID_H)

# Test combinations of Threshold (tau) and Land Gating
thresholds = [0.85, 0.90, 0.92, 0.95]
results = []
eps = 1e-6
```

```
with torch.no_grad():
  for tau in thresholds:
    total_tp = 0
    total_fp_land = 0
    total_fp_sea = 0
    total_gt = 0

    for idx, row in night_test.iterrows():
      data = np.load(row["file_path"])
      raw_img = data["image"]  # [3, 120, 250]
      target = data["mask"]  # [120, 250]
      total_gt += target.sum()

      img_tensor = torch.from_numpy(raw_img).float()
      norm_img = (img_tensor - means) / stds
      padded = (
          F.pad(norm_img, (3, 3, 4, 4), mode="reflect")
          .unsqueeze(0)
          .to(device)
      )

      logits = model(padded)
      probs = torch.sigmoid(logits)[0, 0, 4:124, 3:253].cpu().
         numpy()

      # Base prediction + temperature sanity check
      base_pred = (probs >= tau) & (raw_img[0] >= 285.0)

      # Separate detections into Sea (Lat > 36.9 N ) vs Land (Lat
          <= 36.9 N )
      sea_mask = np.repeat((lat_per_row > 36.9)[:, None], 250,
         axis=1)
      land_mask = ~sea_mask

      pred_land = base_pred & land_mask
      pred_sea = base_pred & sea_mask

      # Accumulate metrics
      total_tp += np.sum((pred_land == 1) & (target == 1))
      total_fp_land += np.sum((pred_land == 1) & (target == 0))
      total_fp_sea += np.sum(pred_sea == 1)

    precision = (total_tp + eps) / (total_tp + total_fp_land +
       eps)
    recall = (total_tp + eps) / (total_gt + eps)
    dice = (2.0 * total_tp + eps) / (
        2.0 * total_tp + total_fp_land + (total_gt - total_tp) +
           eps
    )
```

```
    results.append({
        "tau": tau,
        "tp": int(total_tp),
        "fp_sea": int(total_fp_sea),
        "fp_land": int(total_fp_land),
        "fp_land_per_scene": total_fp_land / len(night_test),
        "recall": recall,
        "precision": precision,
        "dice": dice,
    })

print("\n" + "=" * 95)
print(
    f"{'Threshold':^10} | {'True Pos':^10} | {'FP in Sea':^12} |
       {'FP on Land':^12} | {'FP/Scene (Land)':^16} | {'Recall
       ':^8} | {'Dice':^8}"
)
print("=" * 95)
for r in results:
  print(
      f"{r['tau']:^10.2f} | {r['tp']:^10d} | {r['fp_sea']:^12,d}
         | {r['fp_land']:^12,d} | {r['fp_land_per_scene']:^16.1f}
          | {r['recall']*100:^7.1f}% | {r['dice']:^8.4f}"
  )
print("=" * 95)
```

3. Run the Diagnostic

Run the script:

```
python inspect_night_false_alarms.py
```

This will separate the sea artifacts from genuine land detections across the 100 scenes and identify the exact operating threshold for the night model. Share the table output once it runs.